MDPI

Article

ODFF: Operational Data Foundation Framework

Yoonseok Kang [1] and Dongchul Park [2,*]

[1] Department of Convergence Security, Chung-Ang University, Seoul 06974, South Korea; kangcl1609@cau.ac.kr
[2] Department of Industrial Security, Chung-Ang University, Seoul 06974, South Korea; dongchul@cau.ac.kr
* Correspondence: dongchul@cau.ac.kr

Abstract

Smart manufacturing increasingly depends on continuous and usable operational data, yet in constrained small and medium-sized enterprise (SME) factories, the practical bottleneck often lies not in downstream analytics or AI applications but in the absence of an operationally viable data foundation. This study addresses that problem by proposing an *Operational Data Foundation Framework* for constrained SME manufacturing environments. The framework is derived from a lifecycle-based problem analysis and formalizes six core requirements: continuity, governance, diagnosability, operability, reprocessability, and evolvability. These requirements are further translated into design principles and operational invariants, thereby providing a structured basis for both design and assessment. To examine its field relevance, the framework was instantiated in a real SME deployment involving heterogeneous environmental sensors, a retrofitted legacy scale, LoRa-based field communication, edge-side persistence, cloud-side stream processing, and a governed canonical event model. The evaluation was conducted from the perspective of operational viability rather than benchmark performance alone. Requirement-based evidence showed that the implemented system preserved controlled semantics, supported segmented observability, maintained replayable historical records, and remained manageable under constrained deployment conditions. A representative field case further demonstrated the framework's practical value: LoRa reception instability was diagnosed through segment-wise discrepancy analysis and improved from approximately 33% to 95% packet reception after targeted intervention. The study contributes a field-grounded design logic for operational data systems that can support monitoring, traceability, analytics, alerting, and future extensions in digitally constrained manufacturing settings.

Keywords: smart manufacturing; SME digitalization; industrial data infrastructure; data governance; edge–cloud architecture

1. Introduction

Smart manufacturing is transforming manufacturing from a fixed and fragmented production environment into a connected operational system in which sensing, communication, computation, and decision making are tightly intertwined. Technologies such as the Internet of Things (IoT), cyber–physical systems (CPS), artificial intelligence and machine learning (AI/ML), real-time monitoring, predictive maintenance, and digital twins exemplify this shift. In this context, manufacturing systems can no longer be understood solely as physical production arrangements; they must also be understood as data-mediated operational systems.

Within this transition, data are no longer merely retrospective records. They have become a core operational resource that enables visibility, traceability, quality control,

Received:
Revised:
Accepted:
Published:

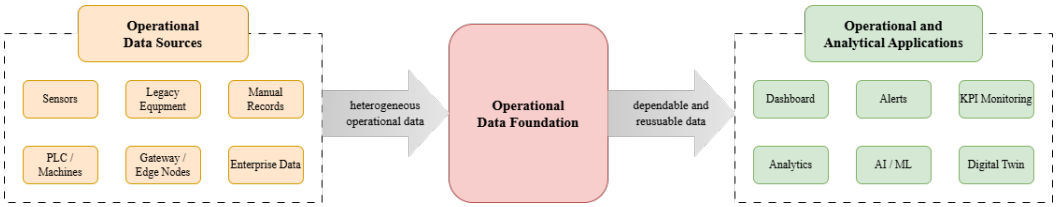

Figure 1. Position of the operational data foundation in a digitalized manufacturing system.

maintenance, optimization, and data-driven decision making. The practical advancement of smart manufacturing increasingly depends on whether operational data can be captured, interpreted, and reused in a stable and sustained manner across the shop floor. In this sense, smart manufacturing capability is closely tied to the continued availability of usable operational data.

At the same time, higher-level intelligent functions do not stand on their own. Predictive analytics, digital twins, and AI-driven optimization all presuppose an underlying operational layer that can continuously capture field data, preserve their meaning across system boundaries, and support downstream reuse. The central challenge, therefore, is not only what manufacturing systems can do more intelligently, but also which data conditions must first be maintained so that such intelligence becomes operationally viable.

The value of manufacturing data is not realized at the moment of generation alone. Rather, it emerges across an end-to-end flow that spans generation, acquisition, transmission, transformation, storage, and downstream use. For this reason, manufacturing data problems cannot be reduced to the performance of a single database, interface, or analytics component. They should instead be understood as operational systems problems distributed across the entire data lifecycle.

This challenge is intensified by the intrinsic heterogeneity of industrial data environments, where different devices, protocols, timestamps, machine states, manual records, and fragmented platforms coexist. Such conditions give rise to structural risks including missingness, duplication, delay, semantic inconsistency, timestamp ambiguity, and source-side variability. In manufacturing settings, these are not minor data imperfections. They directly affect traceability, weaken operational visibility, distort decision making, and ultimately reduce operational efficiency.

What is required, therefore, is not merely an ingestion infrastructure, but an operational layer capable of sustaining data continuity, governed semantics, durable persistence, observability, and downstream reusability. In this study, we refer to this layer as the *operational data foundation*. The core issue is thus not simply whether data can be collected, but whether they can be maintained as an operationally usable and reusable resource. From this perspective, the operational data foundation should not be understood as a mere conduit for data movement. Rather, it provides the basis on which heterogeneous field data can be stabilized, governed, and made dependable for sustained operational and analytical use. As illustrated in Figure 1, it occupies the intermediate layer between heterogeneous operational data sources and higher-level operational and analytical applications.

Although building dependable manufacturing data systems is challenging in general, its foundational requirements become especially visible in constrained small and medium-sized enterprise (SME) factories. These environments rarely offer the conditions often assumed in digitally mature smart-manufacturing settings, such as standardized interfaces, stable internal connectivity, dedicated technical support, and integrated data infrastructure. Instead, they more often combine legacy equipment, fragmented systems, mixed communication technologies, manual records, limited technical staffing, and strong cost sensitivity. For this reason, SME factories should not be regarded merely as smaller-scale plants, but

as critical design settings in which the hidden assumptions of smart manufacturing data systems are exposed most clearly.

Under such conditions, the practical bottleneck of digitalization lies not simply in deploying more sensors, collecting more data, or adding downstream applications. Rather, it lies in whether operational data can be captured with continuity, stabilized with interpretable meaning, and sustained as a usable resource despite fragmentation, instability, and limited operational slack. In SME manufacturing, the central difficulty is therefore not merely data scarcity, but the fragility of the operational data lifecycle through which field data must become dependable for ongoing operational use.

This implies that the primary bottleneck of smart manufacturing in constrained SME factories lies not in downstream intelligence, but in establishing an operationally viable data foundation. Accordingly, this study addresses the following problem: *how should an operational data foundation be designed, and what must it guarantee, in order to make smart manufacturing practically viable in constrained SME factories?*

Existing research has addressed manufacturing data systems from several complementary directions. Smart manufacturing studies have primarily focused on higher-level applications and intelligent capabilities, including real-time monitoring, predictive maintenance, digital twins, and adaptive production support. In parallel, research on industrial data infrastructures has examined pipelines, integration architectures, edge–cloud coordination, semantic consistency, governance, and data quality. Related lifecycle-oriented work has emphasized continuity, traceability, reusable linkage, and the preservation of context across stages, while studies on SME digitalization have clarified the realities of constrained deployment, including legacy equipment, fragmented systems, limited staffing, and strong cost sensitivity.

However, these contributions remain only partially integrated from the perspective of operational viability under constrained deployment conditions. Prior work has often addressed applications, architectural components, lifecycle properties, or SME deployment constraints as related but separate concerns, rather than formalizing what a manufacturing data foundation must guarantee in order to remain operationally usable in practice. As a result, there is still limited systems-level articulation of how these interrelated concerns can be brought together into a coherent design basis for constrained SME manufacturing environments.

To address this gap, this study focuses on the foundational layer that smart manufacturing applications presuppose and formalizes it in terms of operational requirements, design logic, and verification basis. More specifically, this study develops an *Operational Data Foundation Framework* for constrained SME factories and examines it through field-grounded implementation and evidence.

Based on this motivation, the study is guided by the following research questions:

- **RQ1.** What operational risks emerge when smart manufacturing data pipelines are deployed under constrained SME factory conditions?
- **RQ2.** What core requirements must an operational data foundation satisfy in such conditions, and how can they be formalized into design principles and operational invariants?
- **RQ3.** How can the proposed framework be instantiated in a real SME manufacturing setting, and how can its practical value be examined through field evidence?

To answer these questions, this study first organizes a general lifecycle model of smart manufacturing data pipelines as an analytical reference. It then maps constrained SME factory conditions onto that lifecycle in order to identify the key operational risks that hinder practical deployment. In response to these risks, the study formalizes the Operational Data Foundation Framework in terms of core requirements, design principles, and operational

invariants. Finally, it instantiates the framework in a real SME manufacturing setting and examines its practical value through requirement-based evidence and representative field observations.

The primary contributions of this research are as follows:

1. Redefining the actual bottleneck of smart manufacturing in constrained SME factories not as a downstream application problem, but as an operational data foundation problem.
2. Deriving the core operational risks that emerge when a general manufacturing data lifecycle is subjected to constrained SME factory conditions.
3. Formalizing the Operational Data Foundation Framework (ODFF), structured around core requirements, design principles, and operational invariants.
4. Validating the framework's field-grounded operational relevance through real-world implementation in an SME manufacturing environment.

The remainder of this paper is organized as follows. Section 2 reviews the related literature. Section 3 analyzes the general data pipeline lifecycle, SME-specific constraints, and the resulting operational risks. Section 4 presents the proposed Operational Data Foundation Framework. Section 5 describes the field implementation in an SME manufacturing setting. Section 6 evaluates the framework using requirement-based evidence and field observations. Finally, Section 7 concludes the paper.

2. Related Work

2.1. Smart Manufacturing Applications

Smart manufacturing research has primarily advanced through higher-level applications and intelligent capabilities, including industrial IoT, cyber–physical systems, digital twins, real-time monitoring, predictive maintenance, AI/ML-enabled decision making, and adaptive or reconfigurable production support [1–4]. Across these streams, the dominant objective has been to make manufacturing systems more connected, responsive, adaptive, and intelligent, with particular emphasis on operational visibility, data-driven control, and monitoring-to-action linkage [1,2,5].

Despite differences in scope, these application areas share a common prerequisite: continuous or near-real-time operational data from the shop floor [6–8]. Predictive maintenance, real-time monitoring, and digital-twin-based optimization can function effectively only when physical states are continuously captured, aligned with their digital counterparts, and rendered in interpretable form for downstream use [6,9,10]. Several digital twin studies explicitly note that real-time synchronization and high-fidelity input data are essential to the practical realization of intelligent manufacturing functions [4,6,9].

However, much of this literature treats the supporting operational data layer largely as an assumed background condition rather than as a primary object of analysis [1,11]. As a result, prior research has been more explicit about what smart manufacturing applications can optimize, predict, visualize, or automate than about what kind of operational data basis must first be sustained for those functions to remain viable in practice [1,2,6]. For the present study, this gap is important because higher-level smart manufacturing capabilities ultimately depend on whether operational data can be captured, interpreted, and maintained as a usable resource under real deployment conditions [6,9].

2.2. Industrial Data Infrastructures

A substantial body of research has addressed the industrial data basis through data pipelines, ingestion architectures, industrial integration frameworks, streaming and event-driven systems, edge–cloud coordination, and time-series infrastructures. These studies examine how manufacturing data are collected, transmitted, transformed, stored, and used

across field, edge, and cloud environments, as well as how heterogeneous sources can be connected to support downstream services.

Within this literature, source integration and architectural coordination have received sustained attention. Prior work has addressed heterogeneous devices, mixed protocols, fragmented systems, mapping and transformation logic, normalization, interoperability, canonical schemas, reusable interfaces, and scalable coordination structures. These studies make clear that manufacturing data systems are inherently multi-source and multi-layered rather than single-system arrangements.

Another important stream has focused on data quality, semantics, and governance. Dimensions such as accuracy, completeness, consistency, timeliness, and validity have long been treated as central to reliable industrial data use. In manufacturing settings, semantic consistency, metadata, source identity, timestamp interpretation, validation, and monitoring are equally important, since data must remain both available and interpretable.

Recent work has also increasingly recognized that industrial data infrastructures must be managed as ongoing operational systems. Discussions of pipeline management, issue diagnosis, quality monitoring, and maintenance burden show that industrial data systems are not simply built once and then left unchanged. Even so, much of the literature still concentrates on components, flows, or quality properties in isolation, rather than formalizing what such infrastructures must guarantee during operation as an integrated framework.

2.3. Lifecycle Connectivity

Manufacturing data are increasingly understood not as static analytical inputs but as lifecycle-wide resources spanning generation, acquisition, transformation, storage, use, and reuse. In line with this view, prior research has expanded toward lifecycle connectivity, cross-stage information linkage, reusable data layers, digital thread concepts, and traceability-oriented structures. This line of work emphasizes that the value of manufacturing data depends not only on storage or transmission, but also on continuity and connectedness across operational stages.

Lifecycle-oriented studies have particularly emphasized continuity, traceability, and context preservation. Research on digital threads and related concepts has highlighted the importance of linking data across design, production, operation, and maintenance stages, while other studies have stressed persistent identifiers, semantic linkage, and reusable records. Together, these works show that manufacturing data should remain interpretable and reusable beyond their point of initial generation.

Nevertheless, this literature has focused mainly on linkage, visibility, and traceability. Comparatively less attention has been given to how continuity degradation, diagnosability, operability, and reprocessability should be jointly addressed under constrained deployment conditions. This leaves a gap between lifecycle connectivity as a conceptual goal and the operational guarantees required to sustain it in practice.

2.4. SME Deployment

The literature on SME digitalization repeatedly shows that small and medium-sized enterprise factories operate under conditions that differ substantially from those assumed in idealized smart manufacturing architectures. Legacy equipment, fragmented systems, mixed communication technologies, manual records, and heterogeneous devices are common, while stable integration, clean data access, and unified infrastructure are often absent. SMEs should therefore not be understood simply as smaller versions of large factories, but as environments in which digitalization assumptions are most directly challenged.

Prior studies also highlight organizational and operational constraints, including limited technical staffing, the absence of dedicated data or platform teams, reliance on manual troubleshooting, and high maintainability burden. This means that even technically sound architectures may remain difficult to operate and sustain in practice. In SME settings, the question is not only whether a system can be engineered, but whether it can remain operationally viable under realistic staffing and maintenance conditions.

Cost sensitivity, modular adoption, phased rollout, replaceability, and practical maintainability further shape design choices in these environments. Accordingly, SME-related research makes clear that constrained deployment is not a peripheral case, but a central condition under which manufacturing data systems must function.

3. Problem Analysis

This section analyzes the data basis of smart manufacturing as an operational pipeline and examines how constrained SME factory conditions impose structural fragility across that pipeline. The purpose is not merely to describe deployment difficulty, but to show how lifecycle-wide constraints converge into a small set of recurring operational risks. In this way, the section establishes the problem structure that motivates the *Operational Data Foundation Framework* presented in the next section.

3.1. General Data Pipeline Lifecycle

To analyze how manufacturing data become operationally usable, a common analytical reference is first required. In smart manufacturing, data do not derive their value from generation alone; they acquire practical meaning through a structured flow in which field data are captured, transferred, transformed, stored, and connected to downstream use. For this reason, this study adopts a *general data pipeline lifecycle* as an analytical frame rather than as a normative architecture.

In this study, the lifecycle is organized as follows: *data sources* → *data ingestion* → *data transmission* → *data preprocessing* → *data loading* → *data storage* → *data sinks*. Monitoring and management are treated not as a terminal stage, but as cross-cutting operational activities acting across the lifecycle.

- **Data sources:** the points at which data originate, including sensors, equipment states, manual records, and external systems.
- **Data ingestion:** the intake of data from sources into the pipeline.
- **Data transmission:** the transfer of acquired data to the next processing point such as a gateway, edge node, server, or cloud layer.
- **Data preprocessing:** the cleaning, transformation, normalization, and mapping of raw data for downstream use.
- **Data loading:** the controlled insertion of processed data into the target operational layer or storage system.
- **Data storage:** the durable preservation of data for later access, reuse, reinterpretation, and recovery.
- **Data sinks:** the downstream endpoints that consume stored data, including dashboards, alerts, reporting, analytics, and decision support.

This lifecycle is analytically useful because it makes visible the full path through which manufacturing data become usable. In particular, this study treats *data transmission* as an explicit stage, since in constrained manufacturing environments the field–gateway–edge–server–cloud path is itself an independent surface of instability, delay, and loss rather than an invisible internal link. At the same time, the lifecycle remains insufficient as a design answer on its own. It describes flow, but does not specify what must be preserved for that flow to remain operationally viable. It cannot by itself determine how continuity should be

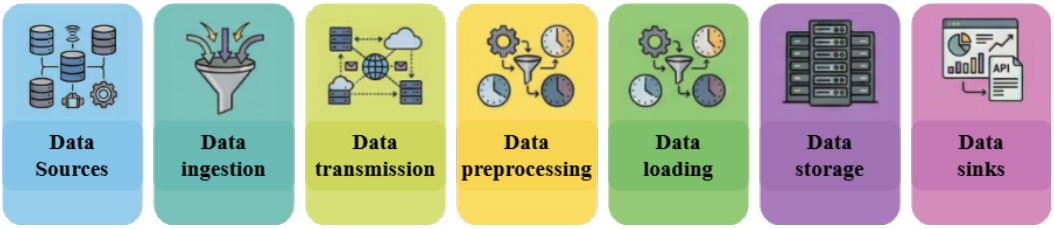

Figure 2. Enter Caption

maintained under segment failure, where semantics must be fixed, how failures should be localized, or how historical records should remain reusable under change. For this reason, the lifecycle is used here as an analytical reference for interpreting constraints and risks rather than as a complete solution model.

3.2. SME Constraints

Constrained SME factories are not simply smaller versions of large digitally mature plants. They are typically shaped by legacy equipment, add-on sensing, mixed interfaces, and fragmented source visibility, which means that heterogeneity begins at the level of *data sources* and continues through ingestion and preprocessing. As a result, the challenge is not only whether values can be collected, but whether diverse formats, units, reporting intervals, and source identities can be brought into a coherent and interpretable structure.

Transmission in SME environments cannot be treated as a neutral internal link. Field data often travel through unstable wireless segments, low-bandwidth paths, battery-powered devices, and gateway- or edge-dependent relay points, making the *ingestion* $\rightarrow$ *transmission* $\rightarrow$ *loading* path a direct source of continuity fragility. Under such conditions, the key issue is not simply whether data were collected at some point, but whether the operational record remains continuous and recoverable despite interruption and delayed recovery.

Manufacturing data also depend on semantic and temporal discipline. In constrained SME settings, heterogeneous preprocessing, ad hoc mappings, implicit transformation logic, inconsistent naming, and multiple unsynchronized clocks can easily destabilize source identity, metric meaning, unit interpretation, and timestamp semantics. These problems become especially acute across *preprocessing* $\rightarrow$ *loading* $\rightarrow$ *storage*, where semantic inconsistency and traceability loss can become structurally embedded. The issue is therefore not merely whether data exist, but whether they can still be read with the same meaning later.

Operational and staffing constraints further weaken lifecycle stability. Many SME factories lack dedicated data or platform teams, and troubleshooting often depends on manual intervention and individual experience. Under such conditions, observability, incident response, maintenance discipline, and ownership clarity tend to be weak across the lifecycle as a whole. This makes it difficult to determine where failures occur, why they recur, and how they can be corrected without increasing long-term operational burden.

Economic and change pressures add a further layer of fragility. SME digitalization typically proceeds under cost sensitivity, lock-in avoidance, modular rollout needs, changing KPI definitions, source onboarding, and ongoing expansion across processes or sites. These pressures accumulate across preprocessing, loading, storage, sinks, and management rather than remaining confined to one stage. The central question is therefore whether the existing structure can absorb change without excessive rework, exception-heavy maintenance, or loss of coherence.

Taken together, these constraints should be understood not as isolated inconveniences but as lifecycle-level design constraints. The core problem in constrained SME factories is

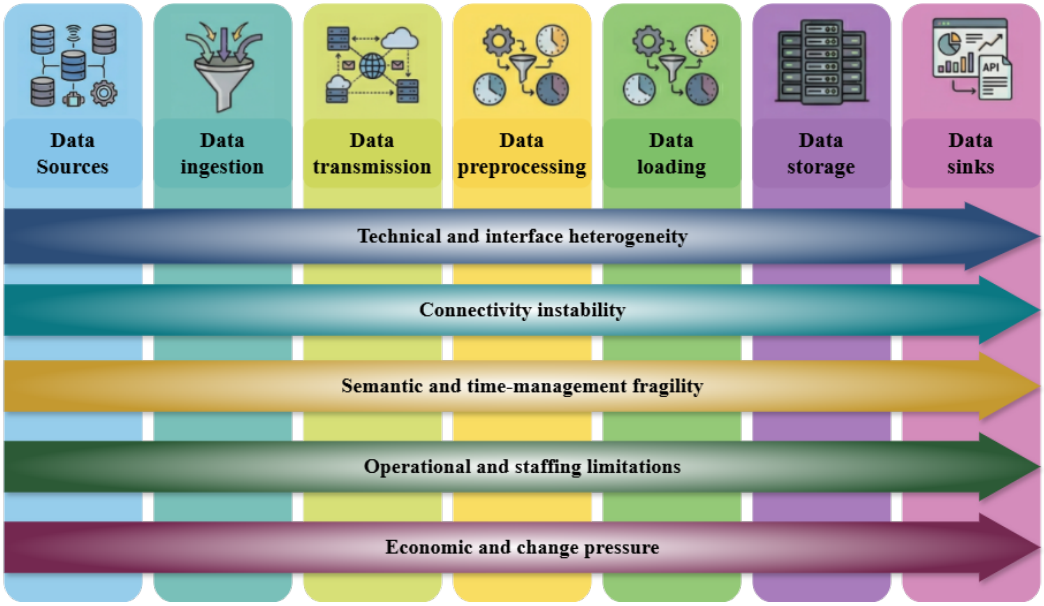

Figure 3. Enter Caption

not merely that deployment is difficult, but that the assumptions of a general data pipeline are systematically destabilized across the lifecycle itself. The next subsection translates this lifecycle-wide fragility into the language of operational risk.

3.3. Operational Risks

The SME constraints described above converge into a smaller set of recurring operational risks. These risks define the problem that the framework in Section 4 must address, because they express not just where deployment becomes difficult, but what the operational data foundation must be able to withstand and preserve.

RSK1. **Continuity risk.** Continuity risk refers to the risk that operational records become interrupted, fragmented, or partially lost because of source instability, fragile ingestion, unstable transmission, or weak persistence discipline. In smart manufacturing, the requirement is not merely that data be collected at a given moment, but that a recoverable and gap-minimized record be maintained over time despite segment-level disruption.

RSK2. **Governance risk.** Governance risk refers to the risk that source identity, metric meaning, timestamp semantics, and transformation logic are not preserved consistently across the pipeline. When heterogeneous preprocessing, implicit semantics, and weak contract discipline prevail, data may remain available while losing stable meaning, thereby weakening downstream interpretation, traceability, and reliability.

RSK3. **Diagnosability risk.** Diagnosability risk refers to the risk that, when a problem occurs, it is difficult to determine where in the lifecycle the failure arose and what failure mode is involved. Weak monitoring, poor stage separation, and low observability turn pipeline degradation into a black-box symptom, making it difficult to localize continuity problems, distinguish semantic failures from transmission failures, and correct recurring issues in a sustained manner.

RSK4. **Operability risk.** Operability risk refers to the risk that the cost and complexity of maintaining, managing, and modifying the system become too high for long-term use. Tight coupling, heavy maintenance burden, unclear ownership, and non-modular structures may still allow a system to function technically, while making it operationally unsustainable under constrained SME staffing and budget conditions.

RSK5. **Reprocessability risk.** Reprocessability risk refers to the risk that historical data cannot be reinterpreted, recomputed, or regenerated when rules, KPIs, schemas, or inter-

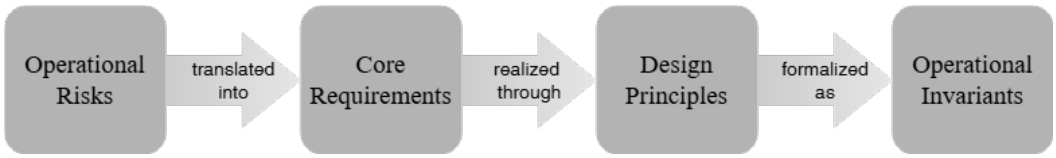

Figure 4. Enter Caption

pretation logic change. When raw preservation is weak, preprocessing is effectively irreversible, and lineage is incomplete, past data lose their value as reusable operational records and become fragile in the face of correction, replay, and reinterpretation.

RSK6. Evolvability risk. Evolvability risk refers to the risk that new sensors, processes, lines, factories, or downstream requirements cannot be incorporated without disproportionate disruption or redesign cost. Because SME digitalization usually proceeds through incremental onboarding rather than one-shot integration, a data structure that cannot absorb change without breaking governance and operational control cannot function as a genuine foundation.

These risks show that a smart manufacturing data pipeline in constrained SME factories cannot be designed as a simple ingestion flow alone. The problem is not a set of isolated device, network, or software issues, but a structural instability of the general data pipeline lifecycle under constrained deployment conditions. What is required, therefore, is an operational data foundation that can explicitly sustain continuity, governance, diagnosability, operability, reprocessability, and evolvability. The next section presents the *Operational Data Foundation Framework* as a structured response to these risks.

4. Operational Data Foundation Framework

4.1. Framework Overview

Section 3 showed that, under constrained SME factory conditions, a smart manufacturing data pipeline cannot be treated as a simple flow of collected data. Rather, it is exposed to recurring risks of continuity loss, semantic inconsistency, weak diagnosability, operational burden, limited reprocessability, and poor evolvability. What is therefore needed is a structured foundation that makes explicit what the data layer must guarantee in order to remain operationally viable.

To address this need, this study proposes the *Operational Data Foundation Framework*. The framework is organized into three layers: *core requirements*, which define the guarantees that the foundation must satisfy; *design principles*, which specify the structural logic needed to realize those guarantees; and *operational invariants*, which define the persistent conditions by which the realized structure can be assessed. Together, these three layers translate the risk structure identified in Section 3 into an explicit and assessable framework for designing and evaluating the operational data foundation.

4.2. Core Requirements

The risks identified in Section 3 show that a smart manufacturing data pipeline must be designed around explicit operational guarantees rather than treated as a simple flow of data across connected components. Accordingly, this subsection defines the *core requirements* of an operational data foundation as the minimum lifecycle-wide conditions required for operational viability under constrained SME factory conditions. These requirements provide the basis for the design principles and operational invariants that follow.

Accordingly, this study defines six core requirements for an operational data foundation in constrained SME manufacturing environments.

R1. Continuity. Data continuity should be preserved as far as practicable despite field disruption and connectivity instability. The essential requirement is not momentary acquisition success, but the minimization of gaps and the preservation of recoverable continuity so that the operational record remains usable even when intermediate segments are unstable.

R2. Governance. Source identity, metric semantics, and time semantics should remain stable across collection, normalization, and storage. Data are not operationally usable merely because they exist; they must retain controlled meaning and interpretable context throughout the pipeline.

R3. Diagnosability. Failures should be observable as segment-level discrepancies rather than only as end-to-end symptoms. The requirement is not simply that problems become visible after damage appears downstream, but that the lifecycle location and mode of failure can be identified in a structurally interpretable way.

R4. Operability. The system should remain maintainable and manageable under constrained staffing and budget conditions. An operational data foundation must therefore be sustainable in day-to-day practice without imposing structurally excessive maintenance burden or operational complexity.

R5. Reprocessability. Historical data should remain reprocessable for rule changes, KPI revisions, correction, and recovery. Manufacturing data should not be treated as one-time consumables, but as records that can be read again, recalculated, and reinterpreted when operational needs evolve.

R6. Evolvability. The foundation should accommodate new sources, processes, sites, and applications without breaking the existing structure. Since SME digitalization typically proceeds through incremental onboarding rather than one-time integration, the data foundation must remain structurally open to change and extension.

4.3. Design Principles

The core requirements defined above specify what the operational data foundation must guarantee, but not how those guarantees should be realized in design. This subsection therefore presents the key *design principles* that translate those requirements into a workable design logic for constrained SME factory conditions. Rather than prescribing specific technologies or implementation choices, these principles define the structural directions needed to sustain the required operational guarantees.

P1. Continuity responsibility should be placed close to the source. If continuity depends primarily on downstream transmission or cloud reachability, it becomes fragile under field disruption and connectivity instability. For this reason, the first responsibility for preserving continuity should be located as close to the source as possible through source-near capture, upstream buffering or persistence, and recovery-oriented ingestion logic.

P2. Raw events should be preserved before abstraction. If only transformed outputs are retained, later replay, reinterpretation, and recovery become structurally limited. The preservation of raw events before irreversible abstraction is therefore a fundamental design principle, since rule changes, KPI revision, and correction all depend on access to preserved pre-abstraction records.

P3. Semantics should be contract-defined. If source identity, metric naming, timestamp semantics, unit definition, and transformation logic are left to ad hoc implementation, semantic inconsistency accumulates across the pipeline. The design must therefore define semantics explicitly through controlled contracts, schema discipline, version-aware rules, and clear identity and time conventions.

P4. Observability should follow pipeline segments. Diagnosability cannot be secured through black-box end-to-end monitoring alone, because operationally meaningful failures must be localizable to specific lifecycle segments. Observability should therefore be structured along the pipeline itself through segment-wise checkpoints, stage-level indicators, and explicit discrepancy visibility.

P5. Responsibilities should be separated by function. If acquisition, normalization, storage, and downstream use are tightly coupled, both maintenance burden and change cost rise rapidly. Functional responsibilities should therefore be separated so that components remain structurally independent, interfaces remain replaceable, and modifications in one part do not unnecessarily destabilize the whole.

P6. Operations should remain lightweight, modular, and open. In constrained SME environments, heavy operational structures and costly maintenance models undermine long-term viability even when technical functionality is achieved. The design should therefore remain lightweight in deployment, modular in composition, manageable in maintenance scope, and open to replacement or extension when operational conditions change.

P7. Change should be treated as a first-class assumption. Changes in KPIs, schemas, rules, source onboarding, and deployment scope are not exceptional events but normal operating conditions in SME digitalization. The foundation should therefore be designed from the outset to tolerate and organize change through version-aware structures, replay support, incremental onboarding discipline, and change-tolerant interfaces.

4.4. Operational Invariants

The core requirements and design principles defined above specify what the operational data foundation must guarantee and how those guarantees should be realized. This subsection completes the framework by defining the *operational invariants*: the persistent conditions that should hold if the foundation is functioning as intended. In this way, the invariants provide the basis for interpreting both the field implementation in Section 5 and the evidence assessment in Section 6.

I1. Durable Capture Invariant. An accepted event should first obtain durable persistence regardless of downstream failure. The operational implication is that the existence of data should not depend on the reliability of later stages, and that continuity should be judged from the point of first durable capture rather than from downstream availability alone.

I2. Traceability Invariant. Every canonical or stored record should remain traceable to its source identity and operational context. The operational implication is that downstream data must remain linked to source identity, contextual metadata, and governing semantics rather than becoming detached from their origin and meaning.

I3. Deterministic Normalization Invariant. The same raw event under the same contract and version conditions should always produce the same canonical output. The operational implication is that normalization must function as a repeatable and controlled semantic transformation rather than as ad hoc interpretation.

I4. Segment-Localizable Failure Invariant. A failure should become visible as at least one discrepancy within a recognizable lifecycle segment. The operational implication is that failure must be observable not merely as a downstream symptom, but as a discrepancy attributable to a specific segment such as source, transmission, preprocessing, loading, storage, or sink.

I5. Replayability Invariant. Preserved raw data should remain reprocessable for rule changes, KPI revision, correction, and recovery. The operational implication is that

historical data must remain an active operational asset rather than a passive archive, capable of supporting later reinterpretation and recomputation.

I6. Modular Replacement Invariant. The replacement of a specific component should not break the semantic chain of the overall foundation. The operational implication is that changes in storage, parsing, transport, or downstream components should not destroy source identity, metric meaning, or replay context across the system.

I7. Onboarding Consistency Invariant. A new source should be incorporable into the existing structure without breaking the prevailing governance discipline. The operational implication is that incremental onboarding should proceed through the same contract, schema, naming, and validation discipline rather than through an accumulation of ad hoc exceptions.

These invariants define the persistent conditions by which the framework can be assessed as operationally realized. If requirements declare the guarantees to be achieved and principles define the design logic needed to achieve them, invariants provide the basis for evaluating whether the realized structure continues to preserve those guarantees in practice.

4.5. Framework Synthesis

The purpose of this subsection is to show that the framework is not an arbitrary set of concepts, but a structured response to the problem space analyzed in Section 3. Section 3 showed how constrained SME factory conditions destabilize the general data pipeline lifecycle and generate recurring risks of continuity, governance, diagnosability, operability, reprocessability, and evolvability. Sections 4.2–4.4 translated those risks into core requirements, design principles, and operational invariants. What remains is to make explicit how these layers connect back to the original problem structure.

Table 1 performs this first synthesis. Its role is to show that constrained SME conditions are not merely a list of deployment difficulties, but recurring operational design problems that can be translated in a structured way. Each major constraint category gives rise to one or more operational risks; each risk is then expressed as a core requirement, translated into one or more design principles, and finally made assessable through corresponding invariants. In this way, the table demonstrates that the framework is derived from recurring structural problems in constrained SME manufacturing rather than assembled as a generic conceptual taxonomy.

Table 1. From SME Constraints to Operational Design Logic.

SME Constraint	Operational Risk	Requirement	Design Principle	Invariant
Heterogeneous devices and fragmented interfaces	RSK2 (Governance risk)	R2 (Governance)	P3 (Semantics must be contract-defined)	I2 (Traceability), I3 (Deterministic normalization)
Add-on sources and partial visibility	RSK2, RSK6 (Evolvability risk)	R2, R6 (Evolvability)	P3, P7 (Change as a first-class assumption)	I2, I7 (Onboarding consistency)
Unstable connectivity and constrained sensing	RSK1 (Continuity risk)	R1 (Continuity)	P1 (Continuity responsibility must be placed close to the source)	I1 (Durable capture)
Weak cross-segment observability	RSK3 (Diagnosability risk)	R3 (Diagnosability)	P4 (Observability must follow pipeline segments)	I4 (Segment-localizable failure)
Limited staff and high maintenance burden	RSK4 (Operability risk)	R4 (Operability)	P6 (Operations must remain lightweight, modular, and open)	I6 (Modular replacement)
Weak raw-data preservation	RSK5 (Reprocessability risk)	R5 (Reprocessability)	P2 (Raw events must be preserved before abstraction)	I5 (Replayability)
Tight functional coupling across the pipeline	RSK3, RSK4, RSK6	R3, R4, R6	P5 (Responsibilities must be separated by function)	I4, I6
Frequent KPI, rule, schema, and source changes	RSK5, RSK6	R5, R6	P7 (Change must be treated as a first-class assumption)	I5, I7
Budget sensitivity and lock-in concerns	RSK4, RSK6	R4, R6	P6	I6, I7

One particularly important translation shown in Table 1 is the movement from semantic and interface heterogeneity to governance logic. In constrained SME factories, heterogeneous devices, mixed protocols, fragmented interfaces, add-on sensing, and manual records do not merely create diverse formats; they destabilize source identity, metric meaning, unit interpretation, and timestamp semantics. For this reason, the problem is not treated here as a narrow integration issue, but as a governance problem requiring the explicit articulation of *R2 Governance*, *P3 contract-defined semantics*, and the assessable conditions of *I2 Traceability* and *I3 Deterministic Normalization*. The broader implication is that manufacturing data become operationally reliable only when meaning and origin remain stably governed.

A second important translation concerns the movement from change pressure to reprocessability and evolvability logic. SME digitalization rarely proceeds as a one-time integration event; instead, KPI definitions evolve, schemas change, new sensors and lines are added, and operational rules are revised over time. When such change is not assumed in advance, transformed-only persistence and weak lineage make past data difficult to reinterpret and new sources difficult to onboard without exception-heavy modification. The framework therefore interprets this problem through *R5 Reprocessability* and *R6 Evolvability*, supported by *P7 change as a first-class assumption* and assessed through *I5 Replayability* and *I7 Onboarding Consistency*. This clarifies that the framework is oriented not only toward present-time correctness, but toward preserving operational coherence under future change.

Table 2 performs a second synthesis by reconnecting the framework to the general data pipeline lifecycle. Its purpose is not to replace the lifecycle introduced in Section 3, but to

reinterpret each lifecycle stage from the perspective of the operational data foundation. The stage labels remain the same, yet under constrained SME conditions each stage acquires additional operational meaning because it must contribute to specific guarantees rather than serve merely as a processing step. In this way, the lifecycle is preserved as an analytical reference while being enriched by the logic of the framework.

The reinterpretation of the transmission stage is especially revealing. In generic pipeline discussions, transmission is often treated as an implicit internal link within ingestion or system connectivity. Under constrained SME factory conditions, however, the path from field devices to gateways, edge systems, servers, and cloud systems is often exposed to intermittent connectivity, limited bandwidth, battery constraints, delayed recovery, and relay dependence. Transmission therefore becomes an independent operational segment in which continuity and diagnosability are actively tested. For this reason, Table 2 assigns the transmission stage the additional meanings of recoverable transfer and segment-aware discrepancy visibility, linking it directly to *R1 Continuity*, *R3 Diagnosability*, *P1 source-near continuity*, *P4 segment-based observability*, *I1 Durable Capture*, and *I4 Segment-Localizable Failure*. This does not reject the general lifecycle; rather, it clarifies what each stage must additionally guarantee under real manufacturing constraints.

In summary, Table 1 shows how constrained SME conditions are translated into operational design logic, while Table 2 shows how that logic reinterprets the general data pipeline lifecycle. The function of this subsection is therefore not to invent a new lifecycle, but to integrate problem space, framework logic, and lifecycle interpretation into a single analytical structure. This synthesis provides the basis for understanding the field implementation in Section 5 and for interpreting the evidence assessment in Section 6.

Table 2. Mapping the General Lifecycle to the Operational Foundation Logic.

General Lifecycle Stage	Added Operational Meaning	Related Risks	Requirements	Principles	Invariants
Data sources	Stable source identity	RSK2, RSK6	R2, R6	P3, P7	I2, I7
Data ingestion	Source-near continuity and intake control	RSK1, RSK2	R1, R2	P1, P3	I1, I2
Data transmission	Recoverable transfer and visible discrepancies	RSK1, RSK3	R1, R3	P1, P4	I1, I4
Data preprocessing	Contract-defined and deterministic transformation	RSK2, RSK6	R2, R6	P3, P7	I2, I3, I7
Data loading	Controlled persistence with replay context	RSK5, RSK6	R5, R6	P2, P7	I5, I7
Data storage	Replay-ready and replaceable persistence	RSK4, RSK5	R4, R5	P2, P6	I5, I6
Data sinks	Governed downstream reuse	RSK2, RSK5	R2, R5	P3, P7	I2, I5
Monitoring and management across the lifecycle	Lifecycle-wide observability and maintainability	RSK3, RSK4	R3, R4	P4, P6	I4, I6

5. Field Implementation

This section describes how the proposed framework was instantiated in a real SME manufacturing setting. The implementation is presented not as a generic IoT deployment, but as a field realization of an operational data foundation under constrained conditions. The focus is therefore placed on how heterogeneous sources were incorporated, how continuity was secured close to the source, how semantics were stabilized through the processing and storage path, and how the resulting data layer was connected to operational visibility and intervention.

5.1. Plant Context and Sensing Objectives

The empirical setting of this study was an SME factory producing traditional food products, namely the Idang Food yakgwa plant. Its production flow spans multiple stages, including raw-material receiving, weighing, mixing, forming, frying, coating, cooling, inner packaging, detection, outer packaging, and storage. Rather than operating as a single highly integrated automated line, the plant consists of multiple workspaces and process segments whose operational states are spatially distributed. As a result, plant status cannot be understood through observation of a single machine or point in time; it requires distributed state information to be captured and connected across the production flow.

Before digitalization, production and quality management at the plant relied primarily on manual records and worker experience. Although records existed, they were neither consistently structured nor accumulated in forms suitable for retrieval, comparison, or downstream analysis. Process-state information was also fragmented across spaces and

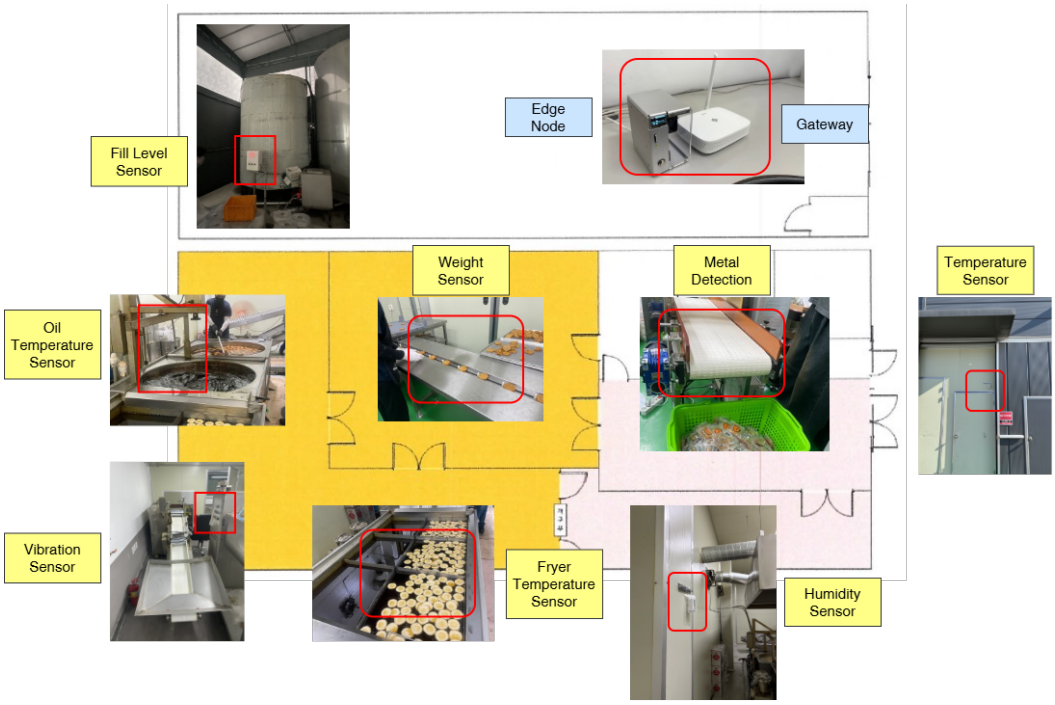

Figure 5. Enter Caption

tasks, making it difficult to trace when and where abnormal conditions emerged and under what environmental or operational changes they occurred. The core problem, therefore, was not the complete absence of data, but the absence of continuous, automatic, and reusable operational records.

This setting also reflects the characteristic constraints of SME manufacturing. The plant did not have a pre-established large-scale smart-factory infrastructure with stable internal connectivity and standardized interfaces. Instead, it combined legacy equipment, fragmented information flow, manual logging practices, and limited internal networking. In particular, always-on WiFi/LAN connectivity was unavailable inside the plant, making direct connection from field devices to a centralized system impractical. In addition, the commercial electronic scale used in production did not provide native networking capability, so its integration required a non-intrusive retrofit rather than equipment replacement. In this sense, the plant was not merely small in scale; it represented a deployment environment in which the assumptions of stable connectivity, standardized access, and centralized visibility were weakened from the outset.

For this reason, the sensing scope of the study was defined not as an arbitrary technology trial, but as a risk-oriented operational design. Temperature and humidity sensors were deployed across key locations including the plant entrance, forming room, heating room, cooling room, drying room, packaging areas, raw-material storage, and finished-product storage. Each placement served a distinct purpose, such as environmental comparison with outside conditions, monitoring of heat-sensitive or humidity-sensitive stages, or preservation tracking during storage. In addition, the inclusion of the electronic scale extended the system beyond an environmental sensor network and brought a legacy production device into the operational data flow. The resulting sensing scope was therefore defined not by what could be measured in general, but by which operational risks and process states needed to be made recordable and traceable.

Taken together, this plant provides an appropriate field setting for the present study because it exposes, in a concentrated form, the conditions under which an operational data foundation becomes necessary. Manual records, fragmented visibility, non-networked

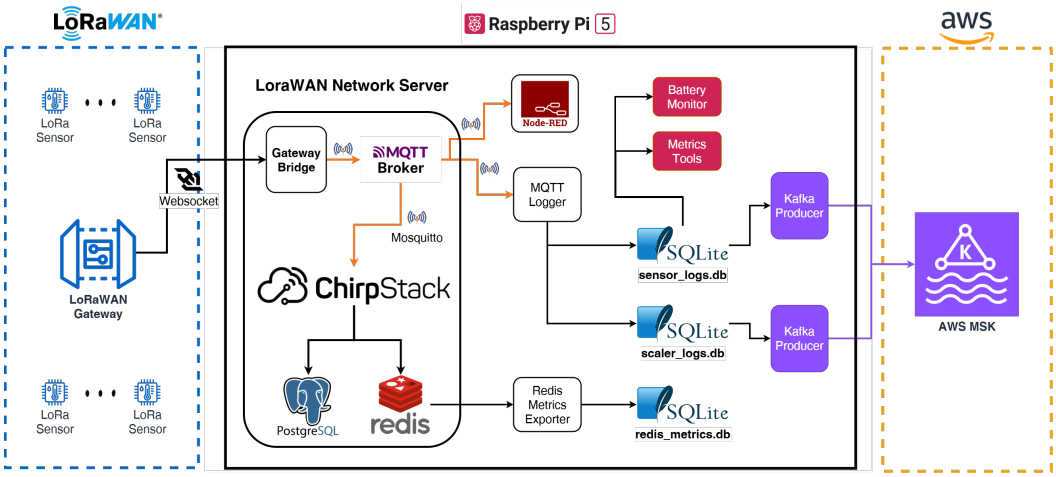

Figure 6. Enter Caption

equipment, limited connectivity, and low operational slack all coexist in this environment. The value of the case lies not in the specific product category itself, but in the fact that it makes visible why an operationally viable data foundation must precede higher-level smart manufacturing applications in constrained SME settings.

5.2. Field Data Acquisition and Edge Stack

To secure field data under these conditions, the acquisition layer was implemented as a source-near architecture in which data were first accepted and preserved locally before being forwarded to upper layers. The overall acquisition path began from source devices such as temperature–humidity sensors and the electronic scale, continued through LoRa communication and a LoRa gateway, and then passed through a ChirpStack-based collection layer, an MQTT broker, and a Raspberry Pi 5 edge node. In this structure, the edge node was not treated as a simple relay point, but as the first operational boundary for accepting, buffering, and preserving field data.

LoRa was selected as the field communication protocol because the plant lacked stable internal Internet infrastructure and required a deployment mode that could support distributed sensing without extensive wiring. The choice was not intended to claim universal superiority over WiFi or LTE, but to support long-range, low-power, and incrementally extensible sensing under brownfield conditions. On this basis, an independent gateway–edge local network was established so that field data could be accepted even in the absence of permanent plant-wide connectivity.

The installed field devices included twelve DRAGINO LHT65N-E3 temperature–humidity sensors, one QW-15 electronic scale, one RAK7268V2 LoRa gateway, and one Raspberry Pi 5 edge server equipped with an NVMe SSD for local persistence. The environmental sensors were battery-powered LoRa devices distributed across the plant, while the gateway and edge node served as the local intake infrastructure. This was therefore not a homogeneous IoT deployment composed only of network-native devices, but a brownfield source environment in which LoRa sensors and a non-networked legacy scale had to coexist within the same operational flow.

The legacy scale required a separate integration path because it did not support native network transmission. To incorporate weighing data without replacing the device, the printer port of the scale was connected through an RS232 cross-serial cable, and the resulting signal was converted into a data stream readable by the Raspberry Pi. In practice, this flow can be summarized as $Scale \rightarrow RS232 \rightarrow RPi \rightarrow LoRa$. In addition, the collection program for the scale was implemented as a daemon script so that data capture resumed

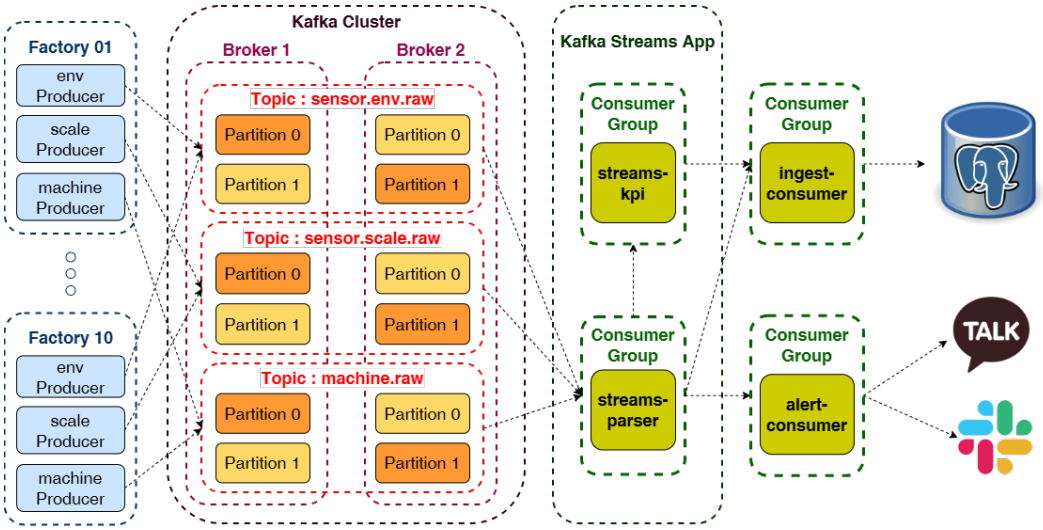

Figure 7. Enter Caption

automatically after disconnection or reconnection. This made the integration operationally usable rather than merely technically possible.

Beyond the gateway, ChirpStack Gateway Bridge and ChirpStack Server managed the LoRaWAN devices and the gateway, while an MQTT broker served as the hub for uplink message transfer. On the edge node, an MQTT logger and a scale logger recorded sensor uplinks and scale data, respectively, into SQLite as local backups. This meant that raw payloads and weighing logs were preserved at the edge regardless of the state of the upper transmission path. The edge therefore functioned as a local persistence boundary rather than as a transient communication hop, and continuity was secured first at the field side rather than deferred to cloud reception.

The edge stack also incorporated diagnosability-oriented support functions. A Redis metrics exporter transferred session- and uplink-related metrics from Redis into SQLite for longer-term analysis, while a battery monitor tracked voltage-related payload fields to observe sensor energy status. Additional metrics tools were used to cross-check gateway reception, device transmission, and database storage states and to analyze reception quality based on RSSI and SNR. The improvement of packet success rate from roughly 33% to approximately 95% in the field showed that edge-side diagnosability was not a secondary convenience, but an operational component of continuity and maintainability. Overall, the field acquisition and edge stack formed an operational capture layer that could incorporate heterogeneous sources, preserve data locally, and support source-near diagnosis of collection instability.

5.3. Cloud Processing Architecture

On top of the field acquisition layer, the system implemented a cloud-side processing architecture designed to interpret, derive, persist, and serve the collected data. The end-to-end flow connected edge producers, a message broker, parser streams, KPI streams, ingest consumers, a database layer, and downstream dashboard and alert services. Rather than concentrating all responsibilities in a single application, this architecture separated message intake, semantic normalization, derived computation, persistence, and service consumption into distinct functional stages.

A central design objective of this architecture was functional decoupling. Producer-side acquisition was not allowed to depend directly on parser execution, KPI computation speed, or database ingestion performance, because failures in downstream stages should not immediately halt the upstream intake path. For this reason, a brokered event flow was

adopted to decouple acquisition from later processing. The architectural significance of the message broker was therefore not simply that Kafka was used, but that the acquisition path and the processing path were separated so that failure propagation could be reduced and downstream evolution could proceed more independently.

The parser stage consumed raw topics and performed decoding, source mapping, metric unification, timestamp handling, deduplication, and canonical event generation. Raw inputs such as `sensor.env.raw`, `sensor.scale.raw`, and `machine.raw` were transformed into a unified canonical stream. The function of the parser was not to forward device-specific payloads downstream, but to convert heterogeneous source messages into a common semantic form that later stages could consume without direct knowledge of device-specific structure.

After normalization, a dedicated KPI stream consumed canonical events and generated derived indicators such as production quantity and yield. KPI computation was intentionally separated from the parser stage and from query-time dashboard logic so that raw normalization and business-level derivation remained distinct responsibilities. This separation reduced semantic overloading in the parser, isolated derived computation from storage concerns, and preserved the possibility of replay under changed KPI rules.

The outputs of the parser and KPI streams were then delivered to the database through ingest consumers. These consumers were designed primarily as an append-only persistence path rather than as another transformation layer. In other words, semantic interpretation and derived computation remained in the stream layer, while the persistence layer focused on durable storage of already-governed events. This design preserved separation between processing logic and storage logic and supported later reprocessing and reuse.

The cloud architecture also maintained clear deployment boundaries. Shared event contracts were placed in a common module, while producers, parser streams, KPI streams, and ingest services were separated into unidirectional dependencies. Consumer-side components were kept lightweight and stateless where possible, while stateful processing responsibility remained with the broker and stream-processing layers. Taken together, the cloud stack was not merely a path for relaying raw events, but a governed, replay-aware, and service-ready processing architecture built on top of the continuity secured at the field edge.

Table 3. Cloud-side Components and Responsibilities.

Component	Input	Output	Primary responsibility
Producer	Edge-side records/messages	Raw topics	Forward field data to the cloud event flow
Parser stream	Raw topics	Canonical events	Decode, normalize, map sources, and standardize events
KPI stream	Canonical events	KPI events	Derive production- and process-level indicators
Ingest consumer	Canonical events, KPI events	Database records	Persist processed events without adding business logic
Database	Ingested records	Queryable tables/views	Provide persistent storage for dashboards, alerts, and analysis
Dashboard/alert layer	Stored and derived data	Operational interfaces	Present status, alarms, and management views

5.4. Data Model

The data model was designed to provide stable semantics and replayable structure across heterogeneous sources. In a field environment where devices, payload formats, and process contexts differ substantially, exposing raw messages directly to downstream systems would have dispersed interpretation responsibility across the pipeline. For this reason, the implementation introduced a canonical event contract that internalized governance into the data path itself. The data model should therefore be understood not merely as a schema design choice, but as a structural mechanism for semantic control.

Each source was represented through a canonical identity organized around `source_id`, including attributes such as `tenant_id`, `line_id`, `process`, `device_type`, and `metric`. This identifier functioned as a semantic anchor rather than as a simple storage key. It allowed downstream components to operate on controlled source meaning without depending on raw device names or payload-specific fields, while a metadata layer preserved the source context in interpretable form.

Time semantics were also explicitly stabilized. In environments where sensor time, gateway time, network-server time, and receipt time may coexist, aggregation, deduplication, and persistence can become ambiguous if no common temporal reference is fixed. In this implementation, `edge_ingest_time` was used as the common pipeline reference time, while `event_time`, `edge_ingest_time`, and `db_ingest_time` were stored as distinct time axes with different roles. This separation prevented confusion between raw occurrence time, pipeline reference time, and durable persistence time, and supported consistent timestamp interpretation across the entire processing path.

Canonical events were generated not through simple field copying, but through normalization rules that included metric unification, value-type alignment, deduplication, and explicit missing-value handling. Deduplication was performed in the parser stream rather than in producers or at the database layer, using ordering and key-based logic appropriate to the message stream. At the same time, the implementation distinguished true zero values from missing observations and minimized unnecessary `null` propagation

in canonical records. These rules were important not as isolated data-cleaning tricks, but as operational rules for preserving stable downstream meaning.

Storage was organized as a layered structure rather than as a single undifferentiated table space. Canonicalized records were written into raw-domain tables such as `raw.sensor_env`, `raw.sensor_scale`, and `raw.machine`; source interpretation metadata were stored in `meta.source`; and derived KPI events were stored in `derived.kpi`. This organization separated raw-like canonical persistence from business-level derivation and thus preserved the distinction between governed operational records and derived management logic.

The resulting model supported replayability and future change. Because the structure preserved a raw-to-canonical-to-derived chain, parser rules or KPI logic could be revised without collapsing the semantic basis of historical records. The combination of a common event contract, append-only ingest, and layered storage was therefore not a purely implementation-specific choice, but a structural condition for reprocessability and evolvability. At the same time, it provided the persistence basis on which dashboards, alerts, and other operational interfaces could be built.

5.5. Operations and Control

The implementation did not stop at data capture and storage. It was also connected to an operational layer through which plant operators and managers could observe state, receive alerts, and intervene when necessary. In this sense, the system was designed not as a hidden back-end pipeline, but as a data foundation that could be translated into operational visibility and action.

Dashboards provided real-time and cumulative views of temperature, humidity, weight, device status, battery condition, packet reception status, and derived KPIs. These views were not designed as raw-signal viewers alone, but as visual interfaces that reorganized operational data into line charts, status summaries, alert histories, and key figures suitable for day-to-day plant awareness. As a result, operators could inspect process conditions, environmental changes, sensor stability, and production-related indicators without relying solely on manual records.

The system also included an alert path for operational intervention. Threshold violations, prolonged silence from data sources, battery-related conditions, and other abnormal states were delivered as asynchronous alerts so that operators could respond promptly. Alert history was preserved alongside the dashboard layer, allowing visual monitoring and intervention support to remain linked rather than isolated. In this design, alerting was not treated as an accessory to monitoring, but as the path through which recognized states became actionable events.

Beyond immediate visibility, the processed and stored data also supported KPI tracking, reporting, and management-level review. Because raw-domain and derived layers were both preserved, the implementation supported not only real-time state checking, but also historical comparison, accumulation-based reporting, and line- or period-level operational review. The dashboard layer therefore served both operator-level situational awareness and manager-level visibility grounded in the same governed data basis.

The implementation further extended to machine-side control capability. The system was built to support actual power control of machinery in the field, meaning that it was capable not only of observing states and issuing alerts, but also of participating in operational intervention. This does not imply that the study centered on control automation itself; rather, it demonstrates that the operational data foundation had already reached the point where downstream device action could be connected to the same semantic and operational chain.

Overall, the operational layer linked dashboards, alerts, KPIs, and machine-side intervention on top of a single governed data foundation. This makes the implemented system more than a data collection pipeline: it functions as an operational interface through which perception, interpretation, response, and future application expansion can be organized on a common basis. The next section evaluates how this implementation satisfies the requirements and invariants defined in Section 4.

6. Evaluation of the Framework

This section evaluates whether the proposed framework functioned as an operational data foundation in a real manufacturing setting. The evaluation does not focus on conventional system benchmarking or algorithmic accuracy. Instead, it examines how the six framework requirements—continuity, governance, diagnosability, operability, reprocessability, and evolvability—became visible in the implemented system and in its operational history. Accordingly, Section 6.1 summarizes requirement-wise evidence, and Section 6.2 presents a representative field case in which continuity and diagnosability were most directly tested through the diagnosis and improvement of LoRa reception instability.

6.1. Framework Compliance

This subsection assesses the framework in terms of the six core requirements defined in Section 4. Rather than reducing each requirement to a single metric, the assessment draws on multiple forms of evidence, including source identity structure, time semantics, layered persistence, segment-wise observability, intervention history, and historical data retention. Table 4 summarizes the main evidence for each requirement and its significance, while the discussion below interprets these observations in operational terms.

Table 4. Requirement-wise Evidence and Significance.

Requirement	Key evidence	Significance
Continuity	local capture, recovery path, reception improvement	recoverable continuity
Governance	canonical schema, source identity, time semantics	controlled and traceable meaning
Diagnosability	segment-wise metrics, gateway-to-DB discrepancy visibility	localizable and actionable failures
Operability	deployable modular stack, monitoring/control interfaces	manageable field operation
Reprocessability	preserved raw data, layered storage	recomputation and correction
Evolvability	extensible source model, modular interfaces	incremental expansion

Continuity was evaluated not as the perfect arrival of every event, but as the extent to which the event flow remained traceable and recoverable under unstable field conditions. In the implemented system, gateway reception, edge-side logging, downstream ingest, and database persistence could be observed as separate stages, allowing continuity to be interpreted through persisted-to-expected ratios, extended no-data intervals, and before–after improvement patterns. Under an expected daily volume of roughly 48,000 events, early persistence remained near 15,800 events, and some source groups exhibited repeated no-data gaps of 2–5 hours. After communication-related adjustments, persisted events recovered to approximately 45,600 per day, while extended gaps decreased from about 18

cases per day to fewer than two. These observations indicate that continuity in this system was not merely a matter of raw transmission success, but of maintaining a recoverable operational record despite intermediate instability.

Governance was reflected in whether heterogeneous field messages could be preserved and interpreted under a stable semantic discipline rather than being stored as loosely structured payloads. In the implemented system, source identity was organized around `source_id`, while the parser stage fixed metric normalization, timestamp normalization, deduplication, and missing-value handling before downstream use. The distinction among `event_time`, `edge_ingest_time`, and `db_ingest_time` prevented ambiguity between occurrence, pipeline reference, and persistence time. In addition, the separation of raw-like persistence from derived KPI layers preserved a stable semantic boundary between operational records and business-level interpretation. Taken together, these structures show that governance was realized as controlled meaning across collection, normalization, and storage.

Diagnosability was evaluated in terms of whether data loss or delay remained a black-box symptom or could be localized to specific lifecycle segments. Because gateway-visible counts, local logs, parser/consumer stages, and database persistence counts could be compared, the same symptom of "too little data" could be interpreted as distinct failure modes. In some intervals, gateway-visible counts were already well below expectations, directing attention to upstream collection and communication; in other situations, the discrepancy widened between gateway-visible counts and persisted records, indicating the need to inspect downstream processing or storage. This segment-wise visibility was especially important in the LoRa case discussed in Section 6.2, where the main source of degradation could be narrowed to the communication-configuration layer rather than being misread as a parser or database issue.

Operability was reflected not merely in technical feasibility, but in whether the system could be deployed, inspected, tuned, and maintained under limited staffing and maintenance capacity. The implementation separated field-side acquisition, gateway and network services, edge persistence, cloud processing, database storage, and dashboard/alert interfaces by function, reducing unnecessary entanglement among acquisition, normalization, storage, derivation, and presentation. In actual operation, interventions such as rejoin handling, configuration adjustment, logging verification, interval tuning, and packet comparison could be performed at the relevant layer without redesigning the system as a whole. This indicates that the structure remained operationally manageable beyond a laboratory prototype and could be sustained under realistic factory conditions.

Reprocessability was assessed in terms of whether collected data remained available for future reinterpretation, recalculation, correction, and recovery rather than being consumed only by the current dashboard or rule logic. The implemented system avoided transformed-only persistence by separating raw-like event storage, source metadata, and derived KPI layers. As a result, historical records remained available for parser rule revision, KPI redefinition, historical audit, and retrospective reconstruction. The accumulated historical storage reached roughly 0.84–1.57 million rows, indicating that the system functioned not only as a short-term monitoring pipeline but also as a retained operational record base capable of supporting later reprocessing needs.

Evolvability was reflected in whether the structure could accept additional sources, process-specific measurements, monitoring tools, KPI logic, and downstream services without breaking its prevailing governance discipline. Because the system was organized around a common event contract, source identity discipline, parser-stage normalization, and modular responsibility separation, new sources could be incorporated primarily through onboarding rules and parser extensions rather than through full pipeline redesign.

During the project, additional source classes, monitoring tools, derived KPI layers, and dashboard/alert functions were progressively introduced while the overall structure remained intact. This indicates that the framework supported staged expansion rather than one-time integration only.

Taken together, the evidence above shows that the proposed framework was realized not only as a conceptual structure but as an operational one. Among the six requirements, continuity and diagnosability were tested most directly through actual field instability, and their operational relevance becomes especially clear in the representative case discussed next.

6.2. Operational Case: LoRa Reliability Improvement

A field data foundation does not remain stable simply because it has once been installed. In practice, reception quality, device condition, battery state, wireless configuration, and operational usage patterns continue to change over time. For this reason, the value of the proposed framework lies not in whether it delivered perfect performance from the outset, but in whether it enabled emerging failures to be detected, localized, and corrected in an operationally meaningful way. This subsection presents a representative case in which severe reception degradation in periodic uplink collection was diagnosed and improved through the framework's continuity and diagnosability logic.

During the early operation period, the plant-wide event flow failed to maintain the expected level of periodic collection, and reception rates in some source groups dropped to roughly 33%. Under an expected daily volume of around 48,000 events, persisted events remained near 15,800, and repeated no-data intervals of 2–5 hours appeared in some groups. This was not merely a matter of lower communication quality. It directly weakened line-level visibility, reduced confidence in dashboard and alert outputs, and undermined the continuity of the operational record.

Because the implemented system did not rely solely on final database status, this degradation could be interpreted as a segment-wise discrepancy rather than as a vague end-to-end symptom. Gateway-visible counts, local edge logs, parser/consumer stages, and database persistence counts could be examined together. In one representative 24-hour window, gateway-visible events were already only about 16,400, far below expectation, whereas the persistence ratio from gateway-visible events to database records remained around 97–99%. This pattern suggested that the main problem lay upstream of parsing and storage, making it more reasonable to investigate the collection and communication segment first.

Through this segment-wise diagnosis, the dominant cause was gradually localized not to parser failure or database loss, but to reception instability associated with the communication-configuration layer. In particular, the combination of confirmed mode, dependence on downlink acknowledgements, and limited channel availability was interpreted as a major source of degradation. In LoRa environments, confirmed uplinks require network-side downlink ACKs. When many field devices transmit at short intervals, downlink resources can become saturated, ACK delays or omissions accumulate, and retransmissions increase contention. Under such conditions, the visible symptom is uplink instability, but the underlying bottleneck is often downlink saturation. For example, if 60 field devices transmit every 5 minutes, the expected uplink load is about 720 messages per hour; even a 25% ACK miss rate can raise retransmission-induced traffic substantially and worsen congestion.

Once the likely bottleneck had been localized to the communication layer, the response proceeded through targeted intervention rather than unguided trial and error. The main adjustments included relaxation of confirmed-mode dependence, expansion of active

channels, device rejoin, transmission-parameter tuning, and refinement of monitoring intervals. Each change corresponded to a specific bottleneck hypothesis, allowing recovery effects to be interpreted in relation to the observed loss pattern. The concrete configuration changes and their intended effects are summarized in Table 5.

Table 5. Segment-wise Event Counts during Reception Degradation.

Stage / Metric	Expected	Observed	Ratio (%)
Scheduled transmissions	48,000	–	–
Gateway-visible events	48,000	16,400	34.2
Edge/local logged events	16,400	16,120	98.3
Ingested events	16,120	15,980	99.1
DB-persisted events	15,980	15,870	99.3

After these adjustments, collection stability improved markedly. Packet reception increased from approximately 33% to 95%, daily persisted events rose from about 15,800 to 45,600, extended no-data intervals decreased from about 18 cases per day to fewer than two, and average gap duration dropped from roughly 145 minutes to 18 minutes. Time-series plots also showed that dense missing segments visible before adjustment were substantially reduced afterward, while observation regularity and line-to-line consistency improved (Figure 8; Table 6).

The significance of this case extends beyond successful LoRa tuning. It demonstrates that continuity degradation could be made observable, that the dominant cause could be narrowed to a specific lifecycle segment, and that the effects of intervention could be re-evaluated within the same operational structure. Continuity was revealed through degradation and recovery, diagnosability through segment-wise discrepancy visibility, and operability through configuration-based field intervention. Governance also mattered, because packet flow could be compared by source group and time window rather than only through aggregate counts. In this sense, the case shows that the proposed framework functioned not merely as a storage path, but as a problem-solving foundation that enabled unexpected field failures to be translated into operationally actionable improvement.

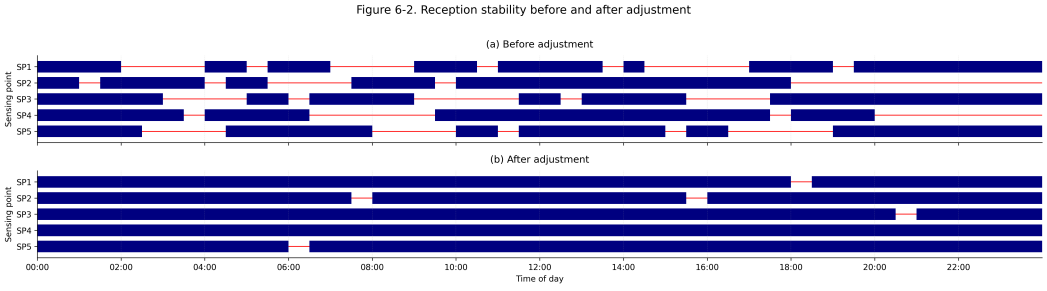

Figure 8. Enter Caption

Table 6. Outcome of Reception Reliability Improvement.

Metric	Before	After	Change
Reception rate (%)	33	95	+62 pp
Persisted events/day	15,870	45,600	+187%
Average gap duration (min)	145	18	–87.6%
Long-gap events/day	18	2	–88.9%
Gateway-to-DB consistency (%)	96.8	99.4	+2.6 pp

7. Conclusion

This study argued that the practical bottleneck of smart manufacturing in constrained SME factories lies not only in downstream analytics or intelligent applications, but more fundamentally in the absence of an operationally viable data foundation. In such environments, legacy equipment, fragmented systems, unstable field conditions, limited staffing, and cost sensitivity weaken the assumptions on which idealized smart manufacturing architectures are often built. The central question, therefore, is not simply how to use more data, but how to design and sustain a data foundation that remains operationally viable under real deployment conditions.

To address this problem, this study proposed the *Operational Data Foundation Framework* for constrained SME factories. The framework formalized six core requirements—continuity, governance, diagnosability, operability, reprocessability, and evolvability—and connected them to corresponding design principles and operational invariants. In this way, the study contributed not merely a particular architecture, but a structured framework that links problem analysis, design logic, and assessment criteria for operational data foundations in smart manufacturing systems.

The framework was then instantiated in a real SME food-manufacturing deployment and evaluated from the perspective of operational viability rather than benchmark performance alone. Through requirement-based evidence and a representative field case, the study showed that the framework remained operationally meaningful in practice. In particular, the diagnosis and improvement of LoRa reception instability demonstrated that continuity and diagnosability were not abstract design goals, but directly relevant to the detection, interpretation, and correction of real operational problems.

More broadly, this study did not seek to add another downstream smart manufacturing application. Instead, it formalized the operational data foundation on which such applications depend. In this sense, the proposed framework provides a reference design logic for building data systems that can support real-time monitoring, traceability, analytics, alerting, and future extensions toward more adaptive and integrated manufacturing systems. Its contribution is therefore both practical and conceptual: it offers a field-grounded way to think about data-system design under constrained deployment conditions.

This study has several limitations. The framework was examined through a single SME food-manufacturing deployment, and its broader generalizability across multiple factories and more diverse industrial settings remains to be tested. In addition, although the implementation supported operational visibility, alerting, and machine-side intervention, the study did not directly evaluate large-scale multi-site deployment, fully automated replay workflows, or closed-loop control integration. Future work may therefore extend the framework through multi-factory replication, richer interoperability settings, automated

regeneration and replay pipelines, and tighter integration with analytics, digital twins, and CPPS-oriented control.

Author Contributions: For research articles with several authors, a short paragraph specifying their individual contributions must be provided. The following statements should be used "Conceptualization, X.X. and Y.Y.; methodology, X.X.; software, X.X.; validation, X.X., Y.Y. and Z.Z.; formal analysis, X.X.; investigation, X.X.; resources, X.X.; data curation, X.X.; writing—original draft preparation, X.X.; writing—review and editing, X.X.; visualization, X.X.; supervision, X.X.; project administration, X.X.; funding acquisition, Y.Y. All authors have read and agreed to the published version of the manuscript.", please turn to the CRediT taxonomy for the term explanation. Authorship must be limited to those who have contributed substantially to the work reported.

Funding: Please add: "This research received no external funding" or "This research was funded by NAME OF FUNDER grant number XXX." and and "The APC was funded by XXX". Check carefully that the details given are accurate and use the standard spelling of funding agency names at https://search.crossref.org/funding, any errors may affect your future funding.

Data Availability Statement: We encourage all authors of articles published in MDPI journals to share their research data. In this section, please provide details regarding where data supporting reported results can be found, including links to publicly archived datasets analyzed or generated during the study. Where no new data were created, or where data is unavailable due to privacy or ethical restrictions, a statement is still required. Suggested Data Availability Statements are available in section "MDPI Research Data Policies" at https://www.mdpi.com/ethics.

Acknowledgments: In this section you can acknowledge any support given which is not covered by the author contribution or funding sections. This may include administrative and technical support, or donations in kind (e.g., materials used for experiments). Where GenAI has been used for purposes such as generating text, data, or graphics, or for study design, data collection, analysis, or interpretation of data, please add "During the preparation of this manuscript/study, the author(s) used [tool name, version information] for the purposes of [description of use]. The authors have reviewed and edited the output and take full responsibility for the content of this publication."

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

MDPI	Multidisciplinary Digital Publishing Institute
DOAJ	Directory of open access journals
TLA	Three letter acronym
LD	Linear dichroism

Appendix A

Appendix A.1

The appendix is an optional section that can contain details and data supplemental to the main text—for example, explanations of experimental details that would disrupt the flow of the main text but nonetheless remain crucial to understanding and reproducing the research shown; figures of replicates for experiments of which representative data are shown in the main text can be added here if brief, or as Supplementary Data. Mathematical proofs of results not central to the paper can be added as an appendix.

Table A1. This is a table caption.

Title 1	Title 2	Title 3
Entry 1	Data	Data
Entry 2	Data	Data

Appendix B

All appendix sections must be cited in the main text. In the appendices, Figures, Tables, etc. should be labeled, starting with "A"—e.g., Figure A1, Figure A2, etc.

References

1. Rozhok, A.; Abate, R.; Manoli, E.; Nele, L. A Review of Recent Advanced Applications in Smart Manufacturing Systems. *J. Manuf. Mater. Process.* **2026**, *10*, 1. https://doi.org/10.3390/jmmp10010001.
2. Bueno, A.; Godinho Filho, M.; Frank, A.G. Smart production planning and control in the Industry 4.0 context: A systematic literature review. *Comput. Ind. Eng.* **2020**, *149*, 106774. https://doi.org/10.1016/j.cie.2020.106774.
3. Cinar, Z.M.; Nuhu, A.A.; Zeeshan, Q.; Korhan, O. Digital Twins for Industry 4.0: A Review. In *Industrial Engineering in the Digital Disruption Era*; Calisir, F.; Korhan, O., Eds.; Lecture Notes in Management and Industrial Engineering, Springer: Cham, 2020. https://doi.org/10.1007/978-3-030-42416-9_18.
4. Atalay, M.; Murat, U.; Oksuz, B.; Parlaktuna, A.M.; Pisirir, E.; Testik, M.C. Digital twins in manufacturing: systematic literature review for physical–digital layer categorization and future research directions. *Int. J. Comput. Integr. Manuf.* **2022**, *35*, 679–705. https://doi.org/10.1080/0951192X.2021.2022762.
5. Song, Y. A Comprehensive Review of Key Technologies and Applications in Smart Manufacturing Systems: From Digital Foundations to Intelligent Applications. In Proceedings of the Proceedings of the 2025 2nd International Conference on Industrial Automation and Robotics, New York, NY, USA, 2025; IAR '25, pp. 328–333. https://doi.org/10.1145/3778886.3778938.
6. Nguyen, P.; Kim, M.; Nichols, E.; Yoon, H.S. AI-Driven Digital Twins for Manufacturing: A Review Across Hierarchical Manufacturing System Levels. *Sensors* **2025**, *26*, 124. https://doi.org/10.3390/s26010124.
7. Tao, F.; Zhang, M. Digital Twin Shop-Floor: A New Shop-Floor Paradigm Towards Smart Manufacturing. *IEEE Access* **2017**, *5*, 20418–20427. https://doi.org/10.1109/ACCESS.2017.2756069.
8. Ding, K.; Chan, F.T.S.; Zhang, X.; Zhou, G.; Zhang, F. Defining a Digital Twin-based Cyber-Physical Production System for autonomous manufacturing in smart shop floors. *Int. J. Prod. Res.* **2019**, *57*, 6315–6334. https://doi.org/10.1080/00207543.2019.1566661.
9. Ojstersek, R.; Javernik, A.; Buchmeister, B. Optimizing smart manufacturing systems using digital twin. *Adv. Prod. Eng. Manag.* **2023**, *18*, 475–485. https://doi.org/10.14743/apem2023.4.486.
10. Qamsane, Y.; Chen, C.Y.; Balta, E.C.; Kao, B.C.; Mohan, S.; Moyne, J.; Tilbury, D.; Barton, K. A Unified Digital Twin Framework for Real-time Monitoring and Evaluation of Smart Manufacturing Systems. In Proceedings of the 2019 IEEE 15th International Conference on Automation Science and Engineering (CASE), 2019, pp. 1394–1401. https://doi.org/10.1109/COASE.2019.8843269.
11. Ciano, M.P.; Pozzi, R.; Rossi, T.; Strozzi, F. Digital twin-enabled smart industrial systems: a bibliometric review. *Int. J. Comput. Integr. Manuf.* **2021**, *34*, 690–708. https://doi.org/10.1080/0951192X.2020.1852600.